

PYTHON AUTOMATION - MODULE WISE LAB REPORT

Selenium • Unit Test Frameworks • BDD & RESTful API Automation • Robot Framework

Prepared By	Rhythm Saha
Enrollment No.	12023052017065
Stream	CSE (IOT CS BT)
Project	Python Automation - Modules 1 to 4 : Lab Report

Submission Note

This report documents the curriculum coverage, core technical concepts, investigation questions, and common implementation challenges for Modules 1 through 4 of the Python Automation training program (Automation with Selenium, Unit Test Frameworks, Python BDD & RESTful API Automation, and Robot Framework), prepared as an individual submission.

CONTENTS

1. Report Purpose
2. Module 1 — Automation with Selenium
3. Module 2 — Unit Test Frameworks (Unittest, PyTest & Page Object Model)
4. Module 3 — Python BDD & RESTful API Automation
5. Module 4 — Robot Framework
6. Overall Conclusion
7. References

1. Report Purpose

This report is prepared as an individual submission covering Modules 1 to 4 of the Wipro Centre of Excellence (CoE) Python Automation training curriculum — Automation with Selenium, Unit Test Frameworks, Python BDD & RESTful API Automation, and Robot Framework. It presents a structured technical-knowledge report for each module: curriculum coverage, core concepts, technical investigation questions, and common implementation challenges with their diagnostic approach, drawn from the official training Table of Contents.

2. Automation with Selenium

2.1 Module Overview

This module lays the foundation for web UI automation using Selenium WebDriver with Python. It covers Selenium's architecture and tooling, environment setup, the full range of element-location strategies, handling both standard and advanced web controls, synchronisation techniques, and the exception-handling and data-handling practices needed to turn a one-off script into a dependable automation asset.

2.2 Core Concepts and Curriculum Coverage

2.2.1 Selenium Architecture — IDE, WebDriver, and Grid

Selenium IDE is a browser extension for recording and replaying simple interactions, useful for quick prototyping but not for scalable frameworks. WebDriver is the core programming interface that sends commands directly to the browser via each browser's native driver, giving full programmatic control. Selenium Grid distributes test execution across multiple machines and browsers in parallel, which is essential once a suite grows beyond a handful of tests.

2.2.2 Environment Setup and WebDriver Installation

A typical setup isolates the project in a virtual environment (venv), installs Selenium via `pip install selenium`, and relies on Selenium Manager — bundled since Selenium 4.6 — to automatically resolve and download the correct browser driver, removing the need to manually manage ChromeDriver/GeckoDriver binaries and PATH entries.

2.2.3 Locator Strategies

Elements can be located by ID, Name, Tag Name, Class Name, Link/Partial Link Text, CSS Selector, or XPath. ID is generally preferred when available and unique; CSS selectors are a fast, readable alternative; XPath (absolute or relative, with `contains()`/`starts-with()`) is the most powerful option for complex or dynamic structures but is also the most brittle if overused.

2.2.4 Handling Web Controls

Standard controls — text fields, buttons, checkboxes, radio buttons, and select/drop-down boxes — are automated through methods such as `send_keys()`, `click()`, `is_selected()`, and the `Select` class. Verifying element state (enabled/disabled, displayed) before interacting with it prevents a large share of flaky-test failures.

2.2.5 Advanced Interactions

Beyond standard controls, real applications require handling JavaScript alerts/confirm/prompts (via `driver.switch_to.alert`), iframes (`driver.switch_to.frame`), multiple windows/tabs (`driver.window_handles`), date pickers, file uploads, drag-and-drop, and dynamic HTML tables that must be traversed row by row.

2.2.6 Synchronisation — Waits

Implicit waits apply a single global timeout to every element lookup, which is simple but imprecise. Explicit waits (`WebDriverWait` combined with `expected_conditions`) block only until a specific, named condition is met — such as visibility or clickability — and are the recommended approach over hardcoded `time.sleep()` calls, which waste time or fail unpredictably.

2.2.7 Exception Handling, Screenshots, and External Data

`try-except-finally` blocks intercept failures (e.g., `NoSuchElementException`, `TimeoutException`) without halting the whole suite, while `driver.quit()` in a `finally` block guarantees the browser process is released. Reading and writing test data from Excel, JSON, and CSV files decouples test logic from hardcoded values, and `driver.save_screenshot()` captures visual evidence at the point of failure.

2.3 Technical Investigation Questions

Q1. What is Selenium, and which kinds of applications is it designed to automate?

Selenium is a browser-automation framework used to control real web browsers programmatically for functional and regression testing of web applications. It is not suited to automating native desktop applications with no browser interface, or to use as a general-purpose backend API client — that role belongs to a library such as `requests`.

Q2. How do Selenium WebDriver, Selenium IDE, and Selenium Grid differ?

`WebDriver` provides the programming interface and direct browser-control mechanism used in scripted automation. `Selenium IDE` is a browser extension for recording and replaying user actions with minimal coding. `Selenium Grid` distributes and parallelises execution of `WebDriver`-based tests across multiple machines, browsers, and operating systems.

Q3. What happens internally when `driver.get()` is called?

The test script's `WebDriver` client sends a navigation command over the `WebDriver` protocol to the browser driver process, which instructs the actual browser to load the URL; once the page reaches the expected ready state, control returns to the script so subsequent commands can execute.

Q4. Why is a virtual environment recommended for a Selenium project?

A virtual environment isolates a project's package versions (Selenium, dependencies) from the system-wide Python installation, preventing version conflicts when different projects on the same machine require different library versions.

Q5. What is Selenium Manager, and what problem does it solve?

`Selenium Manager` is Selenium's built-in driver-management component that automatically detects the installed browser version and downloads a matching driver at runtime, eliminating most manual `ChromeDriver`/`GeckoDriver` version-matching and `PATH` configuration that older Selenium versions required.

Q6. What is the difference between `find_element()` and `find_elements()`?

`find_element()` returns the single first matching `WebElement` and raises `NoSuchElementException` if nothing matches. `find_elements()` returns a (possibly empty) list of all matching elements and never raises for zero matches, which makes it the safer choice when an element's presence is uncertain.

Q7. How do implicit waits, explicit waits, and `time.sleep()` differ, and which is preferred?

An implicit wait sets one global timeout applied to every element lookup; an explicit wait pauses only until a specific condition (such as visibility or clickability) is met for a specific element; `time.sleep()` pauses for a fixed duration regardless of page state. Explicit waits are generally preferred because they wait exactly as long as needed and no longer, avoiding both flakiness and wasted time.

Q8. What is a `StaleElementReferenceException`, and how should it be handled?

It occurs when a previously located `WebElement` is no longer attached to the current DOM, typically because the page re-rendered that section. The correct fix is to re-locate the element after the DOM change (often paired with an explicit wait for the new state) rather than catching and silently ignoring the exception, which could mask a real application defect.

2.4 Common Implementation Challenges and Diagnostic Approach

Challenge	Diagnostic / Resolution Approach
'pip' not recognized as a command	Verify Python was installed with the 'Add to PATH' option checked, and confirm using <code>python --version</code> and <code>python -m pip --version</code> rather than assuming a global pip alias exists.
Package installs successfully but import fails in the IDE	Confirm the IDE/editor's selected interpreter matches the virtual environment where the package was installed; <code>python -m pip show selenium</code> confirms the install location.
Driver/browser version mismatch errors	Let Selenium Manager resolve the driver automatically on a recent Selenium version; if a manually downloaded driver is still on PATH, remove or update it so it cannot conflict with the auto-resolved one.
Browser opens and then closes immediately without completing the script	Check for an unhandled exception terminating the script early, and ensure <code>driver.quit()</code> sits in a finally block rather than being reached only on the success path; review terminal output rather than relying on the visible browser window alone.
'Element not found' despite the element being visible on screen	Confirm the element is not inside an iframe or shadow DOM that requires an explicit context switch, check for dynamically generated IDs, and add an explicit wait for the element's expected state before interacting with it.
Script behaves differently in headless mode	Set an explicit window size in headless mode (viewport defaults can differ), and capture a screenshot or page source on failure to compare rendering against the headed run.

2.5 Key Best Practices

- Prefer explicit waits over implicit waits or `time.sleep()` for anything dynamic.
- Keep locators centralised and prefer stable IDs or CSS selectors over long, brittle XPath chains.
- Always release the driver via `driver.quit()` in a finally block so a failing test cannot leak a browser process.
- Externalise test data (Excel/JSON/CSV) instead of hardcoding values inside the script.

2.6 Module Conclusion

A solid grounding in Selenium's architecture, locator strategies, synchronisation, and exception handling is the prerequisite for everything that follows in the curriculum — the frameworks, API automation, and keyword-driven tooling in later modules all build directly on these fundamentals.

3. Unit Test Frameworks — Unittest, pytest & Page Object Model

3.1 Module Overview

This module builds on raw Selenium scripting by introducing structured, maintainable test-automation frameworks. It covers Python's built-in unittest library, the more feature-rich pytest framework, and the Page Object Model (POM) design pattern that separates page interaction logic from test assertions. Together these form the architectural foundation that most professional QA/SDET teams build their automation suites on.

3.2 Core Concepts and Curriculum Coverage

3.2.1 Unittest Fundamentals

Python's built-in unittest module structures tests as methods inside a TestCase subclass. setUp() and tearDown() run before/after every test method (and setUpClass/tearDownClass once per class), while assertion methods such as assertEquals and assertTrue validate outcomes. Tests are grouped and executed as a TestSuite, and results are summarised through a TestRunner.

3.2.2 pytest Fundamentals

pytest auto-discovers files and functions matching test_*.py / *_test.py without requiring class inheritance. It uses plain assert statements (rewriting them internally for readable failure diffs), supports test selection via -k and -m, and organises shared setup through conftest.py, which any test file in the same directory tree can use without an explicit import.

3.2.3 Fixtures and conftest.py

A fixture is a function decorated with @pytest.fixture that supplies setup/teardown logic and, optionally, a value to the tests that request it as a parameter. Fixture scope (function, class, module, session) controls how often it re-runs. Placing fixtures in conftest.py makes them available project-wide, which is the standard way to manage a WebDriver's lifecycle.

3.2.4 Migrating Selenium Scripts to pytest

Existing Selenium scripts are migrated by moving driver initialisation and driver.quit() into a fixture (typically function- or session-scoped), replacing manual try/finally blocks with pytest's setup/teardown model, and converting assertions to plain assert statements.

3.2.5 HTML and Allure Reporting

The pytest-html plugin (via --html=report.html) generates a self-contained execution report; a pytest_runtest_makereport hook can attach a driver.save_screenshot() call whenever a test's outcome is 'failed', embedding failure evidence directly into the report.

3.2.6 Page Object Model (POM)

POM organises a framework into three layers: Page classes (locators and UI actions only, no assertions), Test classes (test logic and assertions only, no locators), and a BasePage/Utility layer providing shared functionality — such as explicit-wait helpers or CSV/Excel readers — that every page class inherits.

3.2.7 Data-Driven Integration

External data (CSV, Excel via pandas, or JSON) is read in a utility function and fed into tests through pytest's @pytest.mark.parametrize, allowing the same test logic to run once per data row without duplicating code.

3.3 Technical Investigation Questions

Q1. What is the key architectural difference between unittest and PyTest?

Unittest requires tests to be written as methods on a TestCase subclass and uses xUnit-style assertion methods, whereas PyTest allows plain functions with native assert statements, richer fixture-based dependency injection, and more flexible test discovery and parametrization.

Q2. What is a PyTest fixture, and how does conftest.py enable shared setup?

A fixture is a reusable setup/teardown unit that a test requests as a function argument; PyTest resolves and injects it automatically. conftest.py holds fixtures (and hooks) that are automatically visible to every test file in its directory and subdirectories, without any import statement.

Q3. How does @pytest.mark.parametrize enable data-driven testing?

It lets a single test function run once for each tuple of arguments supplied in the decorator, so equivalent test logic executes against many input combinations (e.g., valid/invalid login credentials) without writing repetitive test methods.

Q4. What problem does the Page Object Model solve, and what are its core components?

POM prevents UI changes from breaking every test that touches an element, by centralising locators and interactions in one place. Its core components are the BasePage (shared utilities), individual Page classes (locators/actions for one screen), and Test classes (assertions and test flow).

Q5. How is pytest-html used to generate a report with failure screenshots?

Running `pytest --html=report.html --self-contained-html` produces a single-file report; a `hookimpl(hookwrapper=True)` implementation of `pytest_runtest_makereport` captures a screenshot on failure and attaches it to the report item's 'extra' metadata.

Q6. What is the difference between unittest's setUp/tearDown and PyTest fixture scopes?

setUp/tearDown always run once per test method with no configurable scope. PyTest fixtures can be scoped to function, class, module, or session, so an expensive resource (like a browser session) can be reused across many tests instead of being recreated for each one.

Q7. Why should assertions live only in Test classes and never in Page classes?

Keeping assertions out of Page classes preserves the separation of concerns central to POM: Page classes remain reusable action libraries, while all pass/fail logic — which changes per test scenario — stays in the Test layer, making both easier to maintain independently.

Q8. How is external test data (Excel/CSV/JSON) integrated into a POM framework?

A utility class reads the file (e.g., with `pandas.read_excel` or the `csv` module) and returns a list of records; that list is passed into `@pytest.mark.parametrize` (or a fixture) so each record drives one execution of the same POM-based test.

3.4 Common Implementation Challenges and Diagnostic Approach

Challenge	Diagnostic / Resolution Approach
Fixture not found / not applied to a test	Confirm the fixture is defined in <code>conftest.py</code> at or above the test's directory level, or is explicitly imported; verify the fixture name in the test signature matches exactly.
Flaky tests caused by shared state between tests	Scope fixtures appropriately (function-scoped for mutable resources), avoid module-level globals, and ensure teardown fully resets state (e.g., clearing cookies/local storage) between tests.
Assertions creeping into Page classes	Enforce the POM boundary in code review: Page methods should return values or raise on outright failure to load, but pass/fail judgment belongs in the Test class.
pytest-html report missing failure screenshots	Verify the makereport hook is registered as <code>hookwrapper=True</code> and runs after the 'call' phase; confirm the screenshot path is accessible and correctly attached via the 'extra' report property.
Import errors referencing <code>conftest.py</code> or	Check that <code>conftest.py</code> sits at the correct directory level for its intended

Challenge	Diagnostic / Resolution Approach
page objects	scope, and that the project uses a proper package structure (<code>__init__.py</code> files or a <code>pytest.ini/pyproject.toml</code> <code>rootdir</code>) so imports resolve consistently.

3.5 Key Best Practices

- Keep one Page class per screen/component, with no test assertions inside it.
- Scope fixtures deliberately — function scope for isolation, session scope only for expensive, stateless resources.
- Externalise test data instead of hardcoding values, and drive tests through `parametrize`.
- Always release the `WebDriver` in a fixture's `teardown`/`yield` section so a failing test cannot leak a browser process.

3.6 Module Conclusion

Mastering `unittest`, `PyTest`, and the Page Object Model gives an automation engineer the ability to move from a single working script to a maintainable framework that scales across many test cases, remains resilient to UI changes, and produces clear, shareable execution reports.

4. Python BDD & RESTful API Automation

4.1 Module Overview

This module moves automation below the UI, into the service layer, using Python's requests library to exercise REST APIs directly, and introduces Behaviour-Driven Development (BDD) with Behave so that automated scenarios can be expressed in plain, business-readable Gherkin. Together, these skills let an engineer validate a system's contract independent of its UI and communicate test coverage to non-technical stakeholders.

4.2 Core Concepts and Curriculum Coverage

4.2.1 REST vs. SOAP

REST is an architectural style built on stateless HTTP verbs (GET, POST, PUT, PATCH, DELETE) and typically exchanges JSON; SOAP is a stricter, XML-based protocol with a formal contract (WSDL) and built-in standards for security and transactions. REST's simplicity and lightweight payloads make it the dominant style for modern web/API automation.

4.2.2 Python Requests Library

The requests library wraps HTTP calls in a simple API: `requests.get(url)`, `requests.post(url, json=payload)`, and their PUT/PATCH/DELETE equivalents. Each call returns a `Response` object exposing `.status_code`, `.headers`, `.json()`, and `.text` for validation.

4.2.3 Validating Responses

A response is validated by asserting the expected status code (e.g., 200 for success, 201 for created, 4xx for client errors), checking key headers (such as Content-Type), and asserting on specific fields inside the parsed JSON body rather than the whole payload, to keep tests resilient to unrelated field additions.

4.2.4 PUT vs. PATCH, and Session Management

PUT replaces an entire resource, while PATCH applies a partial update to specific fields. A `requests.Session()` object persists cookies, headers, and TCP connections across multiple calls, which is both more efficient and necessary for flows that depend on server-side session state (e.g., login-then-act sequences).

4.2.5 Authentication and Secrets Handling

Common approaches include Basic Auth, API keys in headers, and Bearer/OAuth tokens attached via the Authorization header. Credentials and tokens should be sourced from environment variables or a secrets manager — never hardcoded in scripts, screenshots, or version control.

4.2.6 Behave and Gherkin

Behave executes `.feature` files written in Gherkin (Given/When/Then), matching each step to a Python function in a `steps` module via decorators and `regex/parse` patterns. This separates the human-readable scenario from its technical implementation.

4.2.7 Hooks, Tagging, and Scenario Outlines

`environment.py` defines hooks such as `before_scenario` and `after_scenario` for setup/teardown (e.g., opening a fresh API session per scenario). `@tag` annotations allow selective execution with `behave --tags=@smoke`, and Scenario Outline with an Examples table drives the same steps across multiple data rows.

4.3 Technical Investigation Questions

Q1. What is the practical difference between PUT and PATCH?

PUT is expected to replace the entire target resource with the payload provided, while PATCH applies a partial, incremental update — only the fields included in the payload are changed, and the rest of the resource is left untouched.

Q2. How does `requests.Session()` improve on individual requests calls?

A Session persists cookies, default headers, and underlying TCP connections (via connection pooling) across multiple requests, which avoids re-authenticating on every call and measurably reduces latency in a multi-step test flow.

Q3. What is Gherkin, and how do Given/When/Then map to Behave step definitions?

Gherkin is a structured, plain-language syntax for describing behaviour. Each Given/When/Then line in a .feature file is matched, via a decorator and pattern, to a corresponding Python function in a steps file that contains the actual implementation for that step.

Q4. What role do `environment.py` hooks play in a Behave framework?

Hooks such as `before_scenario`, `after_scenario`, `before_feature`, and `after_all` let common setup and teardown logic (e.g., initialising an API session or a browser, then cleaning it up) run automatically around every scenario, without repeating that logic in each feature file.

Q5. How does a Scenario Outline with Examples enable data-driven BDD tests?

The same Given/When/Then steps are written once with placeholder variables; the Examples table beneath the Scenario Outline supplies one row of concrete values per execution, so Behave runs the scenario once per row automatically.

Q6. Why do status code ranges (2xx, 4xx, 5xx) matter in API testing?

2xx indicates success, 4xx indicates a client-side error (bad request, unauthorized, not found), and 5xx indicates a server-side failure. Asserting the correct range/exact code confirms the API is both functionally correct and fails predictably under bad input.

Q7. How should API authentication be handled securely in automation code?

Tokens or credentials should be injected from environment variables, a CI secrets store, or a local .env file excluded from version control — attached to requests via the Authorization header — rather than embedded as literal strings in the script.

Q8. Why combine tagging with Behave for CI execution?

Tags let a CI pipeline run a fast `@smoke` subset on every commit while reserving the full regression tag set for scheduled or pre-release runs, balancing fast feedback against thorough coverage.

4.4 Common Implementation Challenges and Diagnostic Approach

Challenge	Diagnostic / Resolution Approach
Intermittent API test failures under load or latency	Add explicit timeouts to requests calls and a bounded retry/backoff strategy; distinguish a genuine defect from transient network conditions before marking a test as failing.
Step definition not matching a Gherkin step (undefined step error)	Run <code>behave --dry-run</code> to list unmatched steps, and verify the regex/parse pattern in the step function exactly covers the wording and parameters used in the feature file.
Hardcoded tokens or credentials in test code	Move secrets to environment variables or a CI secrets manager, and add a pre-commit or CI scan step to catch accidentally committed keys.
Session/auth not reused, causing repeated login overhead	Initialise a <code>requests.Session()</code> once in a <code>before_feature</code> or <code>before_all</code> hook and reuse it for every scenario in that run instead of authenticating per step.
Brittle assertions breaking on unrelated API response changes	Assert only the specific fields relevant to the scenario rather than the full JSON payload, so additive, non-breaking API changes do not fail unrelated tests.

4.5 Key Best Practices

- Assert on specific response fields and status codes, not the entire payload.
- Reuse a Session per logical flow rather than creating a fresh connection for every request.
- Keep Gherkin scenarios business-readable; push technical detail into step definitions, not feature files.
- Never commit API keys, tokens, or passwords — source them from environment variables or a secrets manager.

4.6 Module Conclusion

REST API automation combined with Behave-driven BDD gives an engineer the ability to validate a system below the UI layer and to express that validation in a form both developers and business stakeholders can read, making it a core skill for full-stack test automation roles.

5. Robot Framework

5.1 Module Overview

This module introduces the Robot Framework, a generic, keyword-driven automation framework designed for readability. Rather than writing Python test logic directly, engineers compose test cases from keywords — built-in, library-provided (such as SeleniumLibrary), or custom — making suites approachable to both technical and non-technical team members while still integrating cleanly into CI pipelines.

5.2 Core Concepts and Curriculum Coverage

5.2.1 Robot Framework Architecture

Robot Framework interprets plain-text (or reStructuredText) test files organised into “Settings”, “Variables”, “Test Cases”, and “Keywords” sections. Its core engine is library-agnostic — SeleniumLibrary adds web automation, RequestsLibrary adds API calls — while the syntax and reporting stay identical across libraries.

5.2.2 Test Case Syntax

A test case is a named sequence of keyword calls, each on its own line, optionally passing arguments separated by two or more spaces. Reusable steps are refactored into user-defined `*** Keywords ***`, which themselves are composed of lower-level keywords.

5.2.3 SeleniumLibrary Integration

SeleniumLibrary exposes browser-automation keywords (Open Browser, Input Text, Click Element, Page Should Contain Element) that wrap Selenium WebDriver, giving Robot Framework the same browser-control capability as a native Selenium/Python script, but expressed as keywords.

5.2.4 Data-Driven Testing and Templates

Variables (`${var}`) parameterise values across a suite; a Test Template turns a test case into a reusable pattern executed once per row of supplied arguments — Robot Framework's equivalent of pytest's `parametrize` — reducing duplicated test cases that only differ by input data.

5.2.5 Custom Keywords and Resource Files

Frequently reused keyword sequences are extracted into `.resource` files (or Python-based keyword libraries) and imported with `Resource/Library` statements, allowing multiple suites to share a single, centrally maintained set of actions.

5.2.6 Execution, Tagging, and Parallelisation

`robot suite.robot` runs a suite from the command line; `--include/--exclude` filter execution by tag; the `pabot` tool runs suites or tests in parallel processes, substantially reducing wall-clock time for large regression runs.

5.2.7 Reporting and CI Integration

Every run produces `log.html` (a detailed, keyword-level execution trace) and `report.html` (a high-level pass/fail summary); the `Log` keyword writes custom messages into that trace. Robot Framework's command-line interface and standard exit codes make it straightforward to schedule and gate builds in Jenkins or any CI system.

5.3 Technical Investigation Questions

Q1. What distinguishes Robot Framework's keyword-driven approach from a code-centric framework like pytest?

Test logic is expressed as a sequence of named keywords in a largely natural-language, tabular syntax rather than as Python code, which lowers the barrier for non-programmers to read or author tests while the underlying keywords are still implemented in Python or another supported language.

Q2. What is the purpose of the Settings, Variables, and Keywords sections in a .robot file?

Settings imports libraries/resources and defines suite-level setup/teardown; Variables declares reusable values (URLs, credentials, timeouts); Keywords defines custom, reusable sequences of lower-level keywords that test cases can call by name.

Q3. How does SeleniumLibrary bring browser automation into Robot Framework?

It is a keyword library that internally drives Selenium WebDriver, exposing its capabilities (opening browsers, locating elements, interacting with forms) as Robot Framework keywords such as Open Browser and Click Element, so no direct Selenium Python code is required in the test files.

Q4. What is the difference between a Test Template and a standard data-driven test case?

A Test Template defines the keyword sequence once and then runs it automatically for every row of arguments listed beneath it, whereas a standard test case would need to be duplicated manually for each data variant.

Q5. How are custom keywords organised for reuse across suites?

Common keyword sequences are placed in a .resource file (or a Python keyword library) and imported into any suite via a Resource or Library setting, so the logic is maintained once and shared everywhere it is used.

Q6. How do tags and pabot enable selective and parallel execution?

Tags labelled on test cases let robot --include/--exclude run only a relevant subset (e.g., @regression or @smoke); pabot splits suites or tests across multiple parallel processes, cutting total execution time for large suites.

Q7. What do log.html and report.html provide after a run?

log.html gives a step-by-step, keyword-level trace useful for debugging a specific failure, while report.html gives a concise, suite-level pass/fail summary suitable for a quick status check or stakeholder update.

Q8. How would Robot Framework execution be integrated into a Jenkins pipeline?

A Jenkins build step invokes robot (or pabot) against the suite directory, archives log.html/report.html/output.xml as build artefacts, and Jenkins' post-build actions can be configured to fail the build based on the Robot Framework exit code or parsed results.

5.4 Common Implementation Challenges and Diagnostic Approach

Challenge	Diagnostic / Resolution Approach
'No keyword with name found' errors	Confirm the required Library or Resource is imported in the Settings section and that the keyword name/spelling matches exactly, including expected argument count.
Fragile locators inside SeleniumLibrary tests	Prefer stable IDs or well-formed CSS selectors over deep XPath chains; centralise locators as variables so a UI change requires updating one place.
Duplicated test cases that differ only in input data	Refactor into a Test Template with an Examples-style argument list, eliminating near-identical test cases.
Slow suite execution on large regression runs	Introduce pabot for parallel execution, and tag tests so CI can run a fast subset on every commit and the full suite on a schedule.
Ambiguous keyword name when multiple libraries are imported	Qualify the keyword with its library name (e.g., SeleniumLibrary.Click Element) wherever two imported libraries expose a keyword with the same name.

5.5 Key Best Practices

- Keep test cases at the business-readable keyword level; push technical detail into user-defined keywords or resource files.

- Centralise locators and shared steps in .resource files rather than duplicating them across suites.
- Use Test Templates for data variation instead of copy-pasted test cases.
- Archive log.html, report.html, and output.xml as CI build artefacts for every run.

5.6 Module Conclusion

Robot Framework rounds out the automation skill set with a keyword-driven, highly readable approach that integrates the same underlying Selenium and API capabilities used elsewhere in the curriculum, while remaining accessible to reviewers who do not read Python.

6. Overall Conclusion

Across Modules 1 to 4, the curriculum builds a complete automation skill set in progressive layers: Module 1 establishes the fundamentals of UI automation with Selenium WebDriver; Module 2 introduces structured, maintainable frameworks through PyTest and the Page Object Model; Module 3 extends automation below the UI into REST APIs and business-readable BDD scenarios with Behave; and Module 4 adds a keyword-driven, highly readable framework in Robot Framework that remains accessible to non-programmers while integrating the same underlying automation capability. Together, these modules equip a learner with the UI automation, framework design, service-layer testing, and reporting skills expected of a professional QA/SDET engineer.

7. References

Selenium Documentation: <https://www.selenium.dev/documentation/>

Selenium Manager: https://www.selenium.dev/documentation/selenium_manager/

PyTest Documentation: <https://docs.pytest.org/>

Python unittest Documentation: <https://docs.python.org/3/library/unittest.html>

Python Requests Documentation: <https://requests.readthedocs.io/>

Behave Documentation: <https://behave.readthedocs.io/>

Robot Framework User Guide: <https://robotframework.org/robotframework/>

SeleniumLibrary Documentation: <https://robotframework.org/SeleniumLibrary/>